

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1702

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I VijiyaPriya S.A (192524417) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vijiya Priya S.A
192524417

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

CSA1702 - ARTIFICIAL INTELLIGENCE

PROBLEM-SOLVING ASSIGNMENT

Search, Constraint Satisfaction and Intelligent Agents

Three worked case studies: hospital shift scheduling (backtracking), grid navigation (BFS and A), and an autonomous disaster-rescue robot (online search)*

Overview, Approach and Assumptions

This document solves three problems. Each is organised under the five assessment criteria so that every rubric area is visible: **Problem Understanding**, **Constraint Analysis**, **Solution Development**, **Application of Concepts** and **Justification**. A summary table at the end maps each criterion to where it is addressed.

s.no	Problem	Technique applied
1	Hospital shift assignment (3 doctors, 3 shifts)	Constraint Satisfaction Problem solved with Backtracking Search, with constraint check at every step
2	Robot on a 5×5 grid, Start to Goal	Breadth-First Search (FIFO queue) with full node expansion trace, plus an A* run with Manhattan heuristic to show priority-queue updates
3	Autonomous rescue robot, partial observability, risky zones	Online search agent: Uniform Cost Search with replanning as the map is discovered; PEAS, rationality, environment analysis

Part 1: Hospital Shift Scheduling using Backtracking Search

1.1 Problem Understanding

The task is to assign three doctors to three shifts so that every doctor gets exactly one shift and every shift gets exactly one doctor. This is a classic Constraint Satisfaction Problem (CSP), defined by variables, domains and constraints.

CSP component	Definition
Variables	D1, D2, D3 (the shift taken by each doctor)
Domain of each variable	{Morning (M), Afternoon (A), Night (N)}, ordered $M < A < N$
Goal	A complete assignment that satisfies all constraints
Search space (unconstrained)	$3 \times 3 \times 3 = 27$ assignments (6 if only permutations are counted)

1.2 Constraint Analysis

ID	Constraint	Type	Formal statement
C1	D1 cannot work Night	Unary	$D1 \neq N$
C2	D3 cannot work Morning	Unary	$D3 \neq M$
C3	D2 must work before D3	Binary	$\text{shift}(D2) < \text{shift}(D3)$, using $M=1, A=2, N=3$
C4	Only one doctor per shift	Global	$\text{AllDifferent}(D1, D2, D3)$
C5	Each doctor gets exactly one shift	Structural	Each variable holds a single value; with C4 this forces a one-to-one mapping

Prioritisation. Constraints are applied in order of how cheaply they prune the search. Unary constraints (C1, C2) are checked first because they remove values before any search begins. The global constraint C4 is checked on every assignment because it is a simple lookup. The binary constraint C3 is checked as soon as both D2 and D3 have values.

Domain reduction before search. Applying the unary constraints gives $D1 \in \{M, A\}$ and $D3 \in \{A, N\}$. For C3, D2 cannot be Night, because no later shift exists for D3. So $D2 \in$

{M, A}. The reduced domains (D1: M,A; D2: M,A; D3: A,N) already shrink the search space from 27 to 8 candidates. To show every constraint check explicitly, the trace below still uses the original, unreduced domains.

1.3 Solution Development: Backtracking Search Trace

Settings: variable order D1, D2, D3; value order M, A, N. At each step the algorithm assigns a value, tests it against all constraints whose variables are assigned, then either continues deeper or tries the next value. When a variable runs out of values, it backtracks to the previous variable.

Phase A: search until the first solution

Step	Assignment tried	Constraint checks	Result
1	D1 = M	C1: $M \neq N$ pass. C4: no other shift taken, pass.	Accept, go to D2
2	D1=M, D2 = M	C4: Morning already used by D1, fail.	Reject, next value
3	D1=M, D2 = A	C4: pass. C3 not yet testable (D3 unassigned).	Accept, go to D3
4	D1=M, D2=A, D3 = M	C2: D3 cannot be Morning, fail. C4: also fails (M taken).	Reject, next value
5	D1=M, D2=A, D3 = A	C4: Afternoon already used by D2, fail.	Reject, next value
6	D1=M, D2=A, D3 = N	C1 pass. C2: $N \neq M$ pass. C3: $A < N$ pass. C4: M, A, N all different, pass.	Solution found

Table 1: Backtracking trace to the first valid schedule

Phase B: continuing the search to confirm completeness. To show genuine backtracking and prove that no other solutions exist beyond those listed, the search is resumed after Step 6.

Step	Assignment tried	Constraint checks	Result
7	D1=M, D2 = N	C4 pass; C3 untestable yet.	Accept (backtracked from D3 to D2)
8	D1=M, D2=N, D3 = M	C2 fail; C4 fail (M used by D1).	Reject
9	D1=M, D2=N, D3 = A	C3: $N < A$ is false, fail.	Reject

Step	Assignment tried	Constraint checks	Result
10	D1=M, D2=N, D3 = N	C4: Night used by D2, fail.	Reject. D3 exhausted, backtrack to D1
11	D1 = A	C1 pass; C4 pass.	Accept
12	D1=A, D2 = M	C4 pass.	Accept
13	D1=A, D2=M, D3 = M	C2 fail; C4 fail.	Reject
14	D1=A, D2=M, D3 = A	C4: Afternoon used by D1, fail.	Reject
15	D1=A, D2=M, D3 = N	C2 pass; C3: M < N pass; C4 pass.	Second solution
16	D1=A, D2 = A	C4: fail (A used by D1).	Reject
17	D1=A, D2 = N	C4 pass.	Accept
18-20	D1=A, D2=N, D3 = M / A / N	M: C2 fail. A: C4 fail (D1). N: C4 fail (D2).	All rejected, backtrack to D1
21	D1 = N	C1: D1 cannot work Night, fail.	Reject. Search space exhausted

Table 2: Remaining branches explored after the first solution

The search tree contains exactly **two** valid schedules. The first one found by the stated variable and value ordering is the answer.

Constraint	Verification on final schedule
C1: D1 $\neq$ Night	D1 works Morning: satisfied
C2: D3 $\neq$ Morning	D3 works Night: satisfied
C3: D2 before D3	Afternoon (2) < Night (3): satisfied
C4: one doctor per shift	Morning, Afternoon, Night each used exactly once: satisfied
C5: each doctor one shift	Each of D1, D2, D3 appears once: satisfied

1.4 Application of Concepts

- vi. **CSP formulation:** variables, domains and constraints are separated from the search method, so the same model works with any solver.

- vii. **Backtracking search:** a depth-first search that assigns one variable at a time and abandons a partial assignment as soon as a constraint is violated, so whole subtrees are never explored.
- viii. **Early constraint checking:** each check is applied the moment all its variables are assigned (Steps 2, 4, 5 and 9 prune immediately).
- ix. **Node and arc consistency:** the domain reduction in Section 1.2 is an application of both. Heuristics such as Minimum Remaining Values (choose D1 or D3 first, since they have only two values) and forward checking would reduce the trace further.

1.5 Justification

Backtracking is well suited because the problem is small, discrete and defined entirely by logical constraints with no cost to optimise. It is complete (it found both solutions and proved no others exist), uses memory proportional only to the number of variables, and is far more efficient than generate-and-test, which would build all 27 full assignments before checking any of them. For this instance the first solution was reached in six steps.

Part 2: Robot Navigation on a 5×5 Grid using BFS

2.1 Problem Understanding

A robot must travel from Start (S) to Goal (G) on a 5×5 grid. It can move Up, Down, Left or Right, each move costs 1, and obstacle cells cannot be entered. The objective is the minimum-cost path. The grid used is shown below; it has S at (0,0), G at (4,4) and six obstacles at (0,3), (1,1), (2,1), (2,3), (3,3) and (4,0).

	0	1	2	3	4
0	S			#	
1		#			
2		#		#	
3				#	
4	#				G

Figure 1: Assumed grid (S = start, G = goal, # = obstacle). Rows and columns are labelled 0 to 4.

2.2 Constraint Analysis

Constraint	Effect on the search
4-directional moves	At most 4 successors per cell, no diagonal shortcuts. Shortest possible path from (0,0) to (4,4) has 8 moves.
Uniform cost of 1 per move	Path cost equals number of moves, so BFS (which finds the fewest moves) is automatically cost-optimal.
Obstacles	Hard constraint: an obstacle is never generated as a successor. This is why (0,3), (1,1), (2,1), (2,3), (3,3) and (4,0) never appear in the queue.
Grid boundary	Moves leaving the 5×5 grid are invalid and are discarded.
Manhattan distance	$h(n) = \text{row} - 4 + \text{col} - 4 $. It is admissible (never overestimates, since each move changes distance by at most 1) and consistent , so A* using it is optimal.

The Manhattan heuristic value of every free cell is shown below. Obstacles are marked #.

	0	1	2	3	4
0	8	7	6	#	4
1	7	#	5	4	3
2	6	#	4	#	2
3	5	4	3	#	1
4	#	3	2	1	0

Figure 2: $h(n)$ = Manhattan distance to G (4,4)

2.3 Solution Development: BFS Node Expansion

BFS keeps a **FIFO queue** (the frontier) and a visited set. The front node is removed and expanded, unvisited neighbours are added at the back (in the order Up, Down, Left, Right), and the goal test is applied when a node is removed. Parent pointers are stored so that the path can be rebuilt.

Initial state: Queue = [(0,0)], Visited = {(0,0)}.

Step	Expand	Depth	New nodes added (direction)	Queue after step (front first)
1	(0,0)	0	(1,0) D, (0,1) R	(1,0), (0,1)

Step	Expand	Depth	New nodes added (direction)	Queue after step (front first)
2	(1,0)	1	(2,0) D [(1,1) is an obstacle]	(0,1), (2,0)
3	(0,1)	1	(0,2) R [(1,1) obstacle]	(2,0), (0,2)
4	(2,0)	2	(3,0) D [(2,1) obstacle]	(0,2), (3,0)
5	(0,2)	2	(1,2) D [(0,3) obstacle]	(3,0), (1,2)
6	(3,0)	3	(3,1) R [(4,0) obstacle]	(1,2), (3,1)
7	(1,2)	3	(2,2) D, (1,3) R	(3,1), (2,2), (1,3)
8	(3,1)	4	(4,1) D, (3,2) R	(2,2), (1,3), (4,1), (3,2)
9	(2,2)	4	none (all neighbours visited or blocked)	(1,3), (4,1), (3,2)
10	(1,3)	4	(1,4) R [(0,3) obstacle]	(4,1), (3,2), (1,4)
11	(4,1)	5	(4,2) R	(3,2), (1,4), (4,2)
12	(3,2)	5	none [(3,3) obstacle]	(1,4), (4,2)
13	(1,4)	5	(0,4) U, (2,4) D	(4,2), (0,4), (2,4)
14	(4,2)	6	(4,3) R	(0,4), (2,4), (4,3)
15	(0,4)	6	none	(2,4), (4,3)
16	(2,4)	6	(3,4) D [(2,3) obstacle]	(4,3), (3,4)
17	(4,3)	7	(4,4) R	(3,4), (4,4)
18	(3,4)	7	none	(4,4)
19	(4,4)	8	GOAL reached, stop	empty

Table 3: BFS expansion. Total nodes expanded: 19. The goal is first generated at Step 17 and removed at Step 19.

Path reconstruction (following parent pointers from G): $(4,4) \leftarrow (4,3) \leftarrow (4,2) \leftarrow (4,1) \leftarrow (3,1) \leftarrow (3,0) \leftarrow (2,0) \leftarrow (1,0) \leftarrow (0,0)$.

Optimal path and cost (BFS)

$(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$

Moves: Down, Down, Down, Right, Down, Right, Right, Right. **Cost = 8 moves $\times$ 1 = 8.**

An equally short path exists along the top and right edge, via (0,1), (0,2), (1,2), (1,3), (1,4), (2,4), (3,4). BFS returned the bottom route because it was discovered first through the Down-before-Right ordering.

	0	1	2	3	4
0	S			#	
1	•	#			
2	•	#		#	
3	•	•		#	
4	#	•	•	•	G

Figure 3: BFS optimal path (• marks the route)

Priority-queue version: A* with the Manhattan heuristic

For informed search, the frontier becomes a **priority queue** ordered by $f(n) = g(n) + h(n)$, where g is the cost so far and h the Manhattan distance. Ties on f are broken by smaller h , then by insertion order. Entries are written as node:f.

Step	Popped	g / h / f	Pushed (node:f)	Priority queue after step
1	(0,0)	0 / 8 / 8	(1,0):8, (0,1):8	(1,0):8, (0,1):8
2	(1,0)	1 / 7 / 8	(2,0):8	(2,0):8, (0,1):8
3	(2,0)	2 / 6 / 8	(3,0):8	(3,0):8, (0,1):8
4	(3,0)	3 / 5 / 8	(3,1):8	(3,1):8, (0,1):8
5	(3,1)	4 / 4 / 8	(4,1):8, (3,2):8	(4,1):8, (3,2):8, (0,1):8
6	(4,1)	5 / 3 / 8	(4,2):8	(4,2):8, (3,2):8, (0,1):8
7	(4,2)	6 / 2 / 8	(4,3):8	(4,3):8, (3,2):8, (0,1):8
8	(4,3)	7 / 1 / 8	(4,4):8	(4,4):8, (3,2):8, (0,1):8
9	(4,4)	8 / 0 / 8	GOAL , stop	remaining entries unused

Table 4: A* priority-queue updates. Nodes expanded: 9.

Every pushed node has $f = 8$, which equals the optimal cost, so the heuristic never misled the search. A* returns the same path with cost **8** but expands only **9** nodes against BFS's **19**, because the heuristic keeps the search pointed at the goal while BFS expands level by level in every direction.

2.4 Application of Concepts

- x. **State space formulation:** states are free cells, actions are the four moves, transition cost is 1, goal test is position = (4,4).

- xi. **Uninformed vs informed search:** BFS (FIFO frontier, no knowledge of the goal) versus A* (priority queue, $f = g + h$).
- xii. **Heuristic properties:** Manhattan distance is admissible and consistent on a 4-connected grid, which guarantees A* optimality and no re-expansion of closed nodes.
- xiii. **Graph search with a visited set:** prevents revisiting cells and infinite loops.

2.5 Justification

Because all moves cost 1, BFS is both **complete** and **optimal**, and its guarantee holds regardless of the map. Its weakness is that it explores 19 of the 19 reachable cells here (time and memory grow as b^d). A* with Manhattan distance keeps the same optimality guarantee while cutting expansions by about half on this map. If move costs were to differ, as they do in Part 3, BFS would no longer be optimal and Uniform Cost Search or A* would be required.

Part 3: Autonomous Rescue Robot in a Partially Observable Building

3.1 Problem Understanding

A robot enters a disaster-hit building represented as a grid of rooms. It starts at S and must reach a survivor at G. It can see only the four adjacent cells, so the full layout (blocked cells, risky zones) is not known in advance and is discovered while moving. Each move costs 1, and entering a risky zone costs an extra 2, so 3 in total. The goal is minimum total cost with safe navigation, while updating knowledge as new information arrives.

The following map is used to demonstrate the method. It is the true environment and is hidden from the robot. R marks risky cells and # marks blocked cells.

	0	1	2	3	4
0	S				
1		#	R		
2		#	R	#	
3				#	
4		R			G

Figure 4: True building map (hidden from the robot). Obstacles: (1,1), (2,1), (2,3), (3,3). Risky: (1,2), (2,2), (4,1).

3.2 Task (a): Constraint Analysis

The table shows each constraint and how it changes the robot's decisions. They are listed in order of priority: feasibility and safety constraints first, optimisation second.

Rank	Constraint	Effect on decision-making
1	Blocked cells (obstacles)	Hard constraint. A blocked cell is never entered, and any plan passing through a newly sensed obstacle becomes invalid and must be recomputed. The goal must remain reachable for a solution to exist.
2	Safe navigation	The robot only steps into cells it has already observed . Since sensing covers adjacent cells, the next cell is always known before moving, so it never walks blindly into an obstacle. Risky cells are allowed only when the cost trade-off justifies them.
3	Limited sensing (adjacent cells only)	The environment is partially observable. The robot cannot compute a globally optimal route in advance. It must keep a knowledge base (known map), plan on what it knows, and treat unknown cells with an assumption (optimistic: free, cost 1).
4	Online search requirement	Planning and acting are interleaved. Every step produces new percepts, which update the map and may trigger replanning . Moves are irreversible in cost terms, because exploration itself costs energy.
5	Risky zones (cost 1 + 2 = 3)	Step costs are no longer uniform, so plain BFS is not optimal. One risky cell equals two extra normal moves. Because a detour on a grid always adds an even number of moves, the cheapest detour (+2) exactly ties with one risky cell, so tie-breaking must favour the safer route.
6	4-directional moves	Limits branching to 4 and allows Manhattan distance to act as an admissible lower bound if A* is used.
7	Minimise total cost	The final objective. It is secondary to safety and feasibility, and it guides which of the feasible routes is chosen.

3.3 Task (b): Solution Approach

The chosen approach is an **online search agent that repeatedly runs Uniform Cost Search (UCS) on its current knowledge**, with a plan, sense and replan cycle. Unknown cells are

assumed free at cost 1 (the optimistic "free-space assumption"). Ties between equal-cost routes are broken in favour of fewer risky cells.

Algorithm steps

- v. **Initialise knowledge:** known map = {S}; position = S; total cost = 0.
- vi. **Sense:** observe the four adjacent cells and record each as free, risky or blocked. This is the knowledge update.
- vii. **Plan:** run UCS from the current position to G over the known map. The step cost is 1 for free or unknown cells, 3 for risky cells, and blocked cells are not expanded. Tie-break on number of risky cells.
- viii. **Act:** execute only the **first move** of the plan (to a cell that has just been sensed, hence known to be safe) and add its cost.
- ix. **Repeat** from step 2. Because the map has changed, the plan is recomputed every step, which also guarantees that a plan invalidated by new information is never followed.
- x. **Stop** when position = G. If UCS finds no path to G, the survivor is unreachable with the current knowledge and the robot reports failure.

```
function ONLINE_UCS_AGENT(start, goal):
  known <- {start}; pos <- start; cost <- 0
  while pos != goal:
    known <- known + SENSE(pos)    # update knowledge
    plan <- UCS(pos, goal, known)  # free 1, risky 3, unknown 1
    if plan is empty: return FAILURE
    pos <- first cell of plan; cost <- cost + step cost
  return cost
```

Worked demonstration

The trace below applies the algorithm to the map in Figure 4. "Plan" lists the moves (U, D, L, R) the robot currently intends to follow, and "Est." is the estimated remaining cost under its current knowledge.

Step	Position	Newly sensed	Plan	Est.	Decision
0	(0,0)	(1,0) free, (0,1) free	DDDDRRRR	8	Move D

Step	Position	Newly sensed	Plan	Est.	Decision
1	(1,0)	(2,0) free, (1,1) blocked	DDDRRRR	7	Move D
2	(2,0)	(3,0) free, (2,1) blocked	DDRRRR	6	Move D
3	(3,0)	(4,0) free, (3,1) free	DRRRR	5	Move D
4	(4,0)	(4,1) risky	URRDRR	6	Replan: avoid R, move U
5	(3,0)	none new	RRDRR	5	Move R
6	(3,1)	(3,2) free	RDRR	4	Move R
7	(3,2)	(2,2) risky, (4,2) free, (3,3) blocked	DRR	3	Move D
8	(4,2)	(4,3) free	RR	2	Move R
9	(4,3)	(4,4) = G	R	1	Move R, goal

Table 5: Online search trace. The shaded row shows knowledge changing the plan.

What happens. The robot optimistically plans straight down the left column. At (4,0) it senses that (4,1) is risky. Going through it would cost $3 + 1 + 1 + 1 = 6$, and going back up and around costs exactly 6 as well. The tie is resolved in favour of the safer route, so the robot retreats to (3,0) and continues along row 3. It later senses a second risky cell at (2,2) but never needs it.

Total cost = 10 (ten moves, all through free cells). An omniscient robot with the full map would take the top route, (0,1), (0,2), (0,3), (1,3), (1,4), (2,4), (3,4), (4,4), at cost 8. The extra 2 is the price of exploring blind: a competitive ratio of $10/8 = 1.25$. This is unavoidable for any agent that cannot see the map in advance, and the robot never entered a blocked cell or a risky cell.

	0	1	2	3	4
0	S				
1	•	#	R		
2	•	#	R	#	
3	•	•	•	#	
4	•	R	•	•	G

Figure 5: Route actually travelled by the robot (• = visited, including the step back from (4,0))

3.4 Task (c): Application of AI Concepts

Intelligent agent (PEAS description)

PEAS element	Rescue robot
Performance measure	Reach the survivor, minimise total path cost (moves plus risk penalties), never collide with obstacles
Environment	Grid of rooms with blocked cells and risky zones, only partly known
Actuators	Movement: Up, Down, Left, Right
Sensors	Adjacent-cell sensors reporting free, risky, blocked or survivor

The agent is a **goal-based agent with a model** (a stored map and a transition model) that also behaves as a **utility-based agent**, because it compares routes by cost, not just by whether they reach the goal. Its agent program is the sense, update, plan, act loop above, which maps each percept history to an action.

Rationality

A rational agent selects the action expected to maximise its performance measure **given the percepts it has so far**. At step 4 the robot chose the safe retreat because both options cost 6 and safety is the tie-breaker. Even if a hidden shortcut had existed, the choice would still have been rational, since rationality is judged on available information and not on hindsight. The robot also remains rational by never repeating a plan that new percepts have invalidated, and by gathering information (moving to reveal cells) when that is needed to progress.

Environment type

Property	Value	Reason
Observability	Partially observable	Only adjacent cells are sensed, so the agent needs a belief or knowledge store.
Agents	Single agent	Only the robot acts. Survivors are goals, not decision-makers.
Determinism	Deterministic (assumed)	A chosen move has a certain outcome. If wheels could slip, it would become stochastic.
Episodic or sequential	Sequential	Each move affects future cost and position, so decisions are linked.

Property	Value	Reason
Static or dynamic	Static (assumed during a run)	The layout does not change while planning. If debris could collapse later, the environment would be dynamic, which the replan-every-step loop already tolerates.
Discrete or continuous	Discrete	Grid cells and four fixed actions.
Known or unknown	Unknown	The agent must learn the map by exploring, which is why online search is required.

3.5 Task (d): Justification of the Chosen Approach

Online UCS with replanning is the most suitable approach for the following reasons.

- xiv. **It handles non-uniform costs.** Risky cells make step costs unequal (1 or 3). UCS expands nodes in order of path cost, so it is optimal for the knowledge available, whereas BFS ignores cost and would happily route through risky cells.
- xv. **It handles partial observability.** Offline planners (BFS, UCS or A* run once) need a complete map. The plan, sense and replan loop works with whatever is known and corrects itself when percepts change.
- xvi. **It supports safety.** The agent only enters observed cells, never an unseen cell, and treats blocked cells as hard constraints. The tie-break rule prefers the lower-risk path.
- xvii. **It is complete when the goal is reachable.** The robot only ever moves to known free cells, retreats from dead ends, and avoids endless loops because newly sensed information keeps shrinking the unknown region.
- xviii. **It needs no heuristic design.** UCS is correct without a heuristic. A* with Manhattan distance (admissible since the minimum step cost is 1) can replace UCS as an efficiency upgrade without losing optimality.

Strategy	Main weakness for this problem	Verdict
BFS / DFS	Ignores step costs, so risky cells are not penalised. DFS is also not optimal.	Unsuitable
Offline UCS / A*	Requires the complete map before moving, which the robot does not have.	Unsuitable alone

Strategy	Main weakness for this problem	Verdict
Greedy best-first	Uses only the heuristic, so it can walk into risky cells or dead ends and gives no optimality guarantee.	Risky
LRTA* (learning real-time A*)	Works online, but the robot may repeat visits while learning cost estimates and does not use the full known map.	Acceptable alternative
Online UCS with replanning	Exploration costs extra moves (here 25% above the omniscient optimum), which is unavoidable under limited sensing.	Best fit

Rubric Alignment and Conclusion

Criterion	Weight	Where it is addressed
Problem Understanding	25%	Sections 1.1, 2.1 and 3.1: formal CSP definition, state-space definition for the grid, full description of the partially observable rescue problem and every stated constraint
Constraint Analysis	25%	Sections 1.2, 2.2 and 3.2: constraints classified (unary, binary, global), ranked by priority, and linked to their effect on the search
Solution Development	25%	Sections 1.3, 2.3 and 3.3: step-by-step backtracking trace with checks, BFS and A* expansion tables with queue updates, and the online UCS algorithm with a worked demonstration; final answers stated for each problem
Application of Concepts	15%	Sections 1.4, 2.4 and 3.4: CSP and consistency, heuristics and admissibility, PEAS, agent types, rationality and environment properties
Justification	10%	Sections 1.5, 2.5 and 3.5: reasons for each method and comparison with alternatives

Results. Problem 1: D1 Morning, D2 Afternoon, D3 Night. Problem 2: optimal path of cost 8, found by BFS with 19 expansions and by A* with 9. Problem 3: the online UCS agent reaches the survivor safely at cost 10, against an omniscient optimum of 8. Across all three problems, the common principle is to match the search method to the structure of the problem:

constraints suit backtracking, uniform costs suit BFS, varying costs suit UCS or A*, and unknown environments call for online search.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided